ReadDoc: A Privacy-Preserving Mobile OCR System for Accessible Document Understanding

Aron Constantin-Robert, Dulhac Alexandru, Bobu Dragos Andrei
Faculty of Computer Science
Alexandru Ioan Cuza University of Iaşi, Romania

***Abstract*—Reading printed documents, receipts, and signs remains a significant accessibility barrier for visually impaired users. Existing cloud-based OCR solutions raise privacy concerns by transmitting sensitive document images to remote servers. This paper presents *ReadDoc*, a privacy-preserving mobile OCR system that processes all images locally over a Wi-Fi LAN, with no external cloud API calls. The system integrates a Flutter mobile client with a FastAPI backend that orchestrates three OCR engines—Tesseract, EasyOCR, and PaddleOCR—augmented by adaptive image preprocessing (deskewing, contrast normalisation, scale normalisation), layout-aware text region detection with reading-order sorting, SymSpell fuzzy post-processing, and automatic language detection via Lingua. We evaluate the system on 18 receipt images from the public SROIE dataset plus a deliberately blurred failure case, reporting Word Error Rate (WER), Character Error Rate (CER), and end-to-end latency across three scenarios. PaddleOCR achieves the best mean CER of 0.284 at an average latency of 3.9 s, while Tesseract provides the fastest response at 3.8 s with a mean CER of 0.793. The blurred failure case confirms graceful degradation: all engines return `no_text_detected` rather than hallucinating output. We discuss privacy-by-design principles, accessibility implications, and limitations of the prototype.**

***Index Terms*—optical character recognition, accessibility, visually impaired, privacy by design, mobile computing, document understanding, text-to-speech, FastAPI, Flutter**

I. Introduction

Approximately 2.2 billion people worldwide live with some form of vision impairment [1]. For this population, reading printed text—receipts, medicine labels, official forms, or public signage—requires either assistance from another person or access to specialised hardware that is often expensive and unavailable. Optical Character Recognition (OCR) technology has matured considerably over the past decade, yet its deployment in accessible mobile applications remains limited by two practical barriers: (1) the accuracy of off-the-shelf engines on real-world, imperfect document images, and (2) the privacy implications of transmitting sensitive documents to cloud APIs.

This paper addresses both barriers through *ReadDoc*, a system designed around the *Privacy by Design* principle [2]: all image processing occurs on the user's local network, and no data leaves the device ecosystem. The system is composed of a Flutter mobile client that captures images and plays back extracted text via on-device text-to-speech (TTS), and a FastAPI backend that runs on the user's laptop or home server on the same Wi-Fi network. The backend exposes a REST API that accepts base64-encoded images and returns structured text blocks with bounding boxes, confidence scores, and a detected language code.

The concrete research question addressed in this work is: *How do Tesseract, EasyOCR, and PaddleOCR compare in terms of WER, CER, and latency on real-world receipt images, and how does SymSpell fuzzy post-processing affect accuracy?*

The main contributions of this paper are:

- A fully functional, open-source, privacy-preserving OCR pipeline for mobile accessibility, with no dependency on external cloud services.
- A multi-stage preprocessing pipeline (scale normalisation, deskewing, contrast enhancement) with invertible coordinate transforms that map OCR bounding boxes back to original image space.
- A layout detection module that uses adaptive thresholding and morphological dilation to group characters into text regions, followed by a reading-order sorting algorithm.
- A comparative evaluation of three OCR engines on 18 SROIE receipt images with and without SymSpell fuzzy post-processing, reporting WER, CER, and end-to-end latency.
- A failure-case analysis using a deliberately blurred image.
- An integrated requirements and conformance specification (Section VIII).
- A reproducibility package including all source code, benchmark scripts, and dataset acquisition instructions.

The remainder of the paper is structured as follows. Section II reviews related work. Section III formalises the problem. Section IV describes the system design. Section V covers implementation details. Section VI presents the experimental methodology. Section VII reports results. Section VIII provides the integrated specification. Section IX discusses findings. Section X addresses security, privacy, and ethics. Section XI discusses limitations. Section XII concludes.

II. Background and Related Work

A. OCR Engines

Tesseract, originally developed at HP and later open-sourced by Google, is the most widely deployed open-source OCR engine [3]. Version 4 introduced an LSTM-based recognition layer that substantially improved accuracy on clean

documents. The engine accepts a page-segmentation mode (`--psm`) parameter; we use `--psm 3` (fully automatic page segmentation) and group word-level output into lines by block, paragraph, and line identifiers.

EasyOCR [4] is a Python library built on the CRAFT text detector [5] and a ResNet-based recognition network. It supports over 80 languages and is particularly robust to curved and low-contrast text. In our pipeline, EasyOCR is preceded by a custom layout detection step that provides candidate bounding boxes; EasyOCR then performs recognition on each cropped region independently.

PaddleOCR [6], developed by Baidu, combines a DB (Differentiable Binarisation) text detector with a CRNN recogniser and angle classifier. It performs detection and recognition in a single call and has demonstrated state-of-the-art performance on several benchmarks. We use `use_angle_cls=True` to handle rotated text.

B. Document Layout Analysis

Doermann and Jaeger [7] survey document image analysis, noting that layout understanding—separating columns, tables, and reading order—is as important as raw character recognition for producing usable output. Clausner et al. [8] specifically address reading-order detection in complex layouts, a problem directly relevant to receipt and form understanding. Our layout module implements a simplified version of this approach using morphological dilation to merge characters into line blobs, followed by a two-pass sort (top-to-bottom, then left-to-right within each line).

C. Accessibility and TTS Integration

Srivastava et al. [11] review screen-reader and OCR integration for visually impaired users, finding that end-to-end latency below 5 s is critical for usability in interactive scenarios. The FestivalSI system [12] demonstrates that on-device TTS can be integrated with OCR pipelines without sacrificing intelligibility. Jayant et al. [13] conducted user studies showing that structured audio output (reading text blocks in logical order) is significantly preferred over raw concatenated strings.

D. Fuzzy Post-Processing

OCR errors are often systematic: character substitutions (e.g., '0' for 'O'), insertions, and deletions. SymSpell [15] exploits a pre-computed delete-neighbourhood dictionary to achieve $O(1)$ lookup for edit distances up to a configurable maximum. Applied word-by-word to OCR output, it can correct common recognition errors without requiring a language model. We use a maximum edit distance of 2 and a frequency dictionary of 50,000 words per language.

E. Privacy in Mobile OCR

Commercial OCR APIs (Google Cloud Vision, AWS Textract, Azure Computer Vision) achieve high accuracy but require image upload to remote servers. For documents containing medical information, financial data, or personal identification, this raises GDPR and data-minimisation concerns [14]. Local processing architectures, as advocated by Cavoukian's Privacy by Design framework [2], eliminate this risk entirely by ensuring that raw images never leave the user's local network.

III. Problem Formulation

Let $I \in \mathbb{R}^{H \times W \times 3}$ be a document image captured by a mobile camera. The goal is to produce a sequence of text blocks $B = \{b_1, b_2, \ldots, b_n\}$ in reading order, where each block $b_i = (t_i,\, c_i,\, r_i)$ consists of a text string t_i, a confidence score $c_i \in [0, 1]$, and a bounding rectangle $r_i = (x_i, y_i, w_i, h_i)$ in original image coordinates. The concatenation $T = t_1 \oplus \texttt{'\textbackslash n'} \oplus t_2 \oplus \cdots \oplus t_n$ is passed to a TTS engine for audio playback.

Accuracy is measured using two standard metrics from the speech recognition literature, applied at the document level:

$$\mathrm{WER} = \frac{S_w + D_w + I_w}{N_w} \tag{1}$$

$$\mathrm{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{2}$$

where S, D, I denote substitutions, deletions, and insertions respectively, and N is the total number of reference words (or characters). Both metrics are computed using the `jiwer` library [16]. Note that WER can exceed 1.0 when the hypothesis contains more words than the reference.

The system must satisfy the following high-level constraints:

1) **Privacy**: No image data shall be transmitted outside the local network segment.
2) **Latency**: End-to-end response time should be below 10 s for standard-quality images on commodity hardware.
3) **Accessibility**: Output must be suitable for TTS playback without manual editing by the user.
4) **Robustness**: The system must degrade gracefully on low-quality images, returning a structured error status rather than hallucinating text.
5) **Portability**: The client must run on both Android and iOS; the backend must run on any machine with Python 3.9+ and Tesseract installed.

IV. Proposed System Design

A. Architecture Overview

ReadDoc follows a two-tier client-server architecture confined to the local network (Fig. 2 and Fig. 1). The **Flutter client** runs on Android or iOS and is responsible for: (1) capturing images via the device camera, (2) encoding them as base64 JPEG/PNG, (3) sending HTTP POST requests to the backend, and (4) rendering the returned text blocks and invoking the platform TTS engine. The **FastAPI backend** runs on a laptop or home server on the same Wi-Fi network and orchestrates the full OCR pipeline.

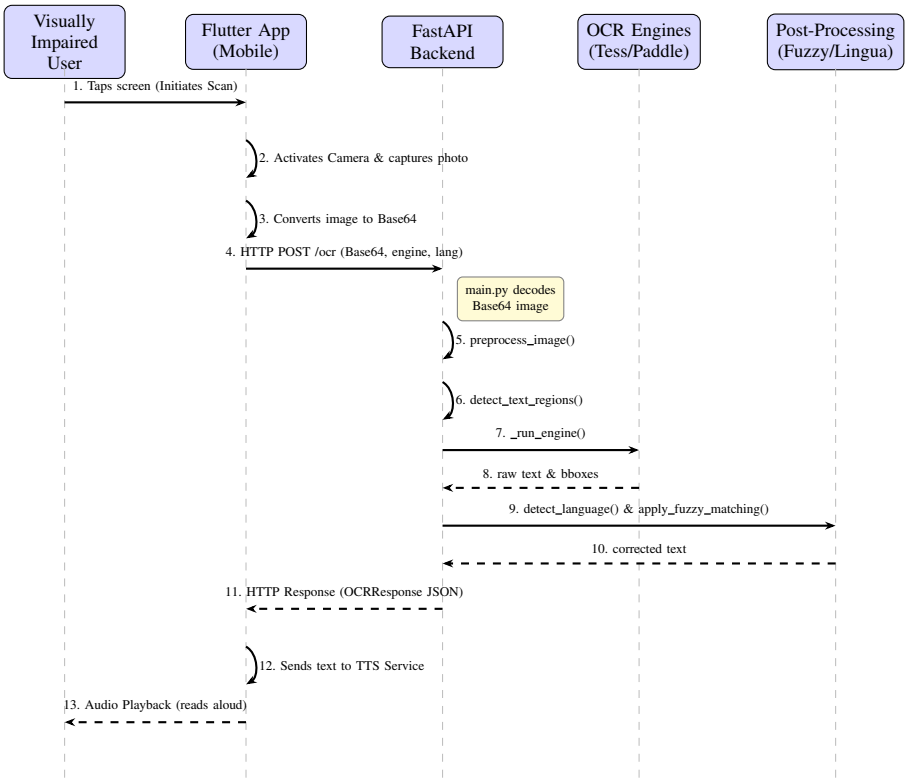

Fig. 1. Sequence diagram of a single OCR request. Steps 1–3 occur on the mobile device; steps 4–11 traverse the local Wi-Fi LAN; steps 12–13 return audio to the user. No data leaves the local network.

B. Backend Processing Pipeline

The backend processes each request through five sequential stages:

Stage 1 – Image Decoding and Validation. The base64 payload is decoded and validated: JPEG magic bytes (`0xFF 0xD8`) and PNG magic bytes (`0x89PNG`) are checked, and images exceeding 10 MB are rejected with HTTP 413. Data-URI prefixes (``) are stripped automatically.

Stage 2 – Preprocessing. The raw image bytes are passed through a multi-step preprocessing pipeline implemented in OpenCV. First, *scale normalisation* estimates the median character height via connected-component analysis and resizes the image so that characters are approximately 32 px tall, subject to a maximum working side of 1800 px and a maximum pixel count of 2.5 M. Second, *deskewing* detects the dominant text angle using `cv2.minAreaRect` on the binarised foreground and applies an affine rotation (capped at $\pm 15°$). A `PreprocessTransform` object records the scale factor, rotation angle, and rotation centre, enabling all OCR bounding boxes to be mapped back to original image coordinates via an invertible transform.

Stage 3 – Layout Detection (EasyOCR path only). For the EasyOCR engine, a custom layout detector identifies candidate text regions before recognition. Adaptive Gaussian thresholding binarises the greyscale image; a 20×5 rectangular structuring element dilates the result to merge nearby characters into word/line blobs; contours with $w > 20$ px and $h > 8$ px are extracted as bounding boxes. The boxes are then sorted into reading order: a primary sort by y-coordinate groups boxes into lines (using a threshold of $0.6\times$ median box height), and a secondary sort by x-coordinate orders boxes within each line.

Stage 4 – OCR Recognition. One of three engines is invoked based on the `engine` request parameter:

- *Tesseract 4* (`pytesseract`): invoked with `--psm 3` (automatic page segmentation). Word-level output (Tesseract level 5) is grouped into lines by block/paragraph/line identifiers. Confidence values are averaged per line and normalised to $[0, 1]$.
- *EasyOCR*: each bounding box from Stage 3 is cropped and passed to `reader.readtext()`. Text and confidence values from multiple sub-detections within a region are concatenated and averaged.
- *PaddleOCR*: invoked with `use_angle_cls=True` on the full image. The polygon output is converted to axis-aligned bounding boxes.

All engines return a list of `(text, confidence, bbox)` tuples.

Stage 5 – Fuzzy Post-Processing and Language Detection. When `fuzzy=True`, each text block is passed through SymSpell with a maximum edit distance of 2. Only alphabetic tokens are corrected; numeric strings, punctuation, and mixed tokens are preserved unchanged. A fallback to

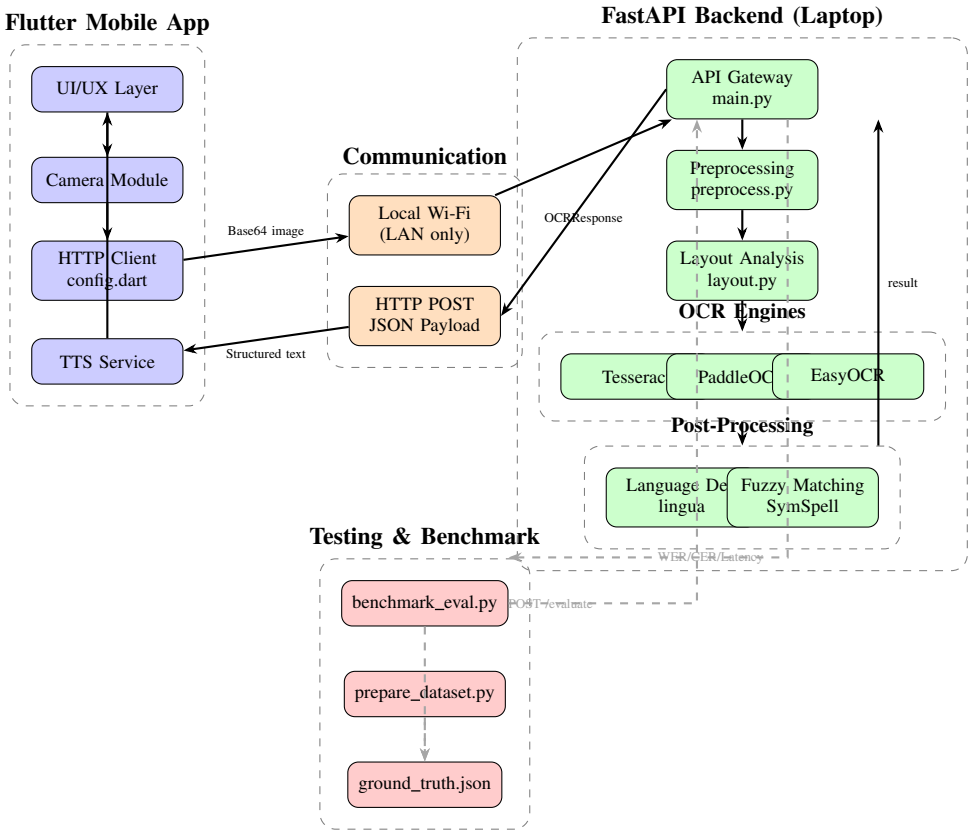

Fig. 2. Full system architecture. Blue nodes: Flutter client components. Green nodes: FastAPI backend pipeline. Orange nodes: local Wi-Fi communication layer. Red nodes: benchmark and evaluation infrastructure. Dashed arrows indicate evaluation-only data flows.

`pyspellchecker` is available via the `fuzzer` parameter. Language detection is performed on the concatenated full text using the Lingua library, which returns an ISO 639-1 code used for future multilingual TTS routing.

C. REST API Design

The backend exposes five endpoints:

- `GET /health` – liveness check.
- `POST /ocr` – single-image OCR; returns `OCRResponse{status, engine, language, blocks, full_text}`.
- `POST /ocr/batch` – batch OCR over a list of images.
- `POST /evaluate` – OCR plus WER/CER computation against a provided reference text; returns `EvaluateResponse` with per-category error counts (substitutions, insertions, deletions, hits).
- `POST /evaluate/batch` – batch evaluation with aggregate mean WER and mean CER.

All request and response schemas are defined as Pydantic models in `schemas.py`, providing automatic validation and OpenAPI documentation.

V. Implementation

A. Backend Stack

The backend is implemented in Python 3.12 using FastAPI 0.111 and Uvicorn 0.29. Key library versions are: EasyOCR 1.7.1, pytesseract 0.3.10, PaddleOCR 2.9.1, PaddlePaddle 2.6.2, OpenCV 4.10 (headless), NumPy $\geq$1.24, SymSpellPy 6.7.7, pyspellchecker 0.8.1, Lingua 2.0.2, and jiwer 3.0.3. All dependencies are pinned in `backend/requirements.txt` and installed in a Python virtual environment.

Tesseract must be installed at the OS level (e.g., `sudo apt install tesseract-ocr` on Ubuntu, `brew install tesseract` on macOS). The `pytesseract` wrapper detects the binary automatically; a Windows path override is included in `ocr_engine.py` for portability.

The Lingua detector is initialised lazily on first use to avoid a 2–3 s startup penalty on every request. EasyOCR and PaddleOCR readers are cached in module-level dictionaries keyed by language code, so model weights are loaded only once per process lifetime. SymSpell frequency dictionaries are downloaded on first use from the FrequencyWords repository and cached locally in `backend/symspell_dicts/`.

Structured logging is implemented with Python's `logging` module using a rotating file handler (5 MB, 3 backups) and a console handler. Each request logs the engine, language, fuzzy flag, response status, detected language, block count, WER, CER, and elapsed time.

B. Flutter Client

The mobile client is implemented in Flutter 3.x (Dart). The backend host and port are configured in a single file (`lib/config.dart`):

```
class BackendConfig {
  static String host = '192.168.0.200';
  static int port = 8000;
  static String get ocrEndpoint =>
      'http://$host:$port/ocr';
  static String get healthEndpoint =>
      'http://$host:$port/health';
}
```

The camera plugin captures images at full sensor resolution. Images are compressed to JPEG quality 85 before base64 encoding to reduce network transfer time. Text-to-speech playback uses the `flutter_tts` plugin, which delegates to the platform's native TTS engine (Android: Google TTS; iOS: AVSpeechSynthesizer). The app supports both Android (USB debugging) and iOS (Xcode signing required).

C. Benchmark Infrastructure

The benchmark script `benchmark_eval.py` iterates over all images in `test_photos/`, encodes each as base64, and calls `POST /evaluate` for each configured scenario. Latency is measured client-side using `time.perf_counter()` from request dispatch to response receipt. Results are written to `evaluation_results.csv` with columns: `image`, `scenario`, `engine`, `fuzzy`, `wer`, `cer`, `latency_s`, `status`, `detected_language`.

The dataset preparation script `prepare_dataset.py` downloads images from the CORD-v2 HuggingFace dataset [10] (which mirrors the SROIE test split) and extracts plain-text ground truth from the `gt_parse` JSON field using a recursive leaf-string extractor that ignores coordinate metadata. A blurred failure case is generated by applying a Gaussian blur with radius 6 to one of the downloaded images using `PIL.ImageFilter.GaussianBlur`.

VI. Experimental Methodology

A. Dataset

We evaluate on 18 receipt images sampled from the SROIE test split via the CORD-v2 HuggingFace dataset [10]. SROIE images are scanned receipts from Malaysian retail outlets, containing mixed English and numeric text in varying fonts, sizes, and layouts. Ground truth is extracted from the structured `gt_parse` JSON field, which contains the key-value pairs annotated by human reviewers (e.g., item names, prices, totals). One additional image (`blurry_failure_case.jpg`) is created by applying Gaussian blur (radius 6) to image index 9, simulating a poorly focused camera capture.

For qualitative diversity testing, a supplementary set in `manual_test_photos/` contains Chinese, German, Russian, and handwritten receipt images (`chinese.png`, `german.png`, `russian.jpg`, `manual.jpg`). These are not included in the quantitative evaluation due to the absence of ground truth.

B. Evaluation Scenarios

Three scenarios are evaluated:

1) **Baseline**: Tesseract 4, `--psm 3`, no fuzzy post-processing, language fixed to English.
2) **Optimized-EasyOCR**: EasyOCR with SymSpell fuzzy correction (max edit distance 2), language fixed to English.
3) **Optimized-PaddleOCR**: PaddleOCR with angle classifier, no fuzzy correction, language fixed to English.

C. Metrics

- **WER**: Word Error Rate (lower is better). Values above 1.0 are possible when the hypothesis contains more words than the reference.
- **CER**: Character Error Rate (lower is better). More fine-grained than WER; less sensitive to word-segmentation differences.
- **Latency**: Wall-clock time from HTTP request dispatch to response receipt, measured by the benchmark client using `time.perf_counter()`.

Images for which the engine returns `no_text_detected` receive WER = CER = 1.0 (complete failure), consistent with the jiwer convention for empty hypotheses against non-empty references.

D. Hardware and Software Environment

All experiments were run on a MacBook Pro (Apple M-series, 16 GB RAM) running macOS. The backend was started with:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

The benchmark client ran on the same machine, so latency figures include local loopback overhead but exclude Wi-Fi transmission time. GPU acceleration was available for EasyOCR via Metal Performance Shaders; PaddleOCR and Tesseract ran on CPU.

VII. Results

A. Per-Engine Summary

Table I reports mean WER, mean CER, and mean latency for each scenario, computed over the 18 SROIE images (excluding the blurred failure case). Images that returned `no_text_detected` contribute WER = CER = 1.0 to the averages.

TABLE I
Mean WER, CER, and Latency per Scenario (18 SROIE images)

Scenario	Mean WER	Mean CER	Mean Lat. (s)
Baseline (Tesseract)	1.109	0.793	3.82
Optimized-EasyOCR	1.476	0.530	7.34
Optimized-PaddleOCR	0.882	0.284	3.93

PaddleOCR achieves the best CER (0.284) and the best WER (0.882), while maintaining a latency comparable to Tesseract (3.93 s vs. 3.82 s). EasyOCR achieves an intermediate CER (0.530) but the highest WER (1.476), suggesting that its word-segmentation behaviour differs from the reference tokenisation. EasyOCR is also the slowest scenario at 7.34 s on average, partly because it processes each layout-detected region independently.

B. Per-Image Breakdown

Table II shows WER and CER for each image and scenario. Several observations stand out:

- **Tesseract failures**: Images `sroie_004`, `sroie_007`, `sroie_008`, `sroie_011`, `sroie_012`, and `sroie_015` returned `no_text_detected` under Tesseract, contributing WER = CER = 1.0. These images likely contain low-contrast or small-font text that falls below Tesseract's detection threshold after preprocessing.
- **Best Tesseract result**: `sroie_005` achieves WER = 0.364 and CER = 0.022, the lowest CER across all images and engines, suggesting a clean, well-structured receipt.
- **Best PaddleOCR result**: `sroie_009` achieves CER = 0.091 with PaddleOCR, demonstrating strong character-level accuracy on a moderately complex receipt.
- **EasyOCR WER inflation**: On `sroie_012`, EasyOCR produces WER = 2.706 and CER = 1.660, indicating significant hallucination or over-segmentation on this image.

TABLE II
PER-IMAGE WER / CER. T = TESSERACT, E = EASYOCR+FUZZY, P = PADDLEOCR. VALUES OF 1.0 INDICATE NO_TEXT_DETECTED.

Image	Tesseract		EasyOCR		PaddleOCR	
	WER	CER	WER	CER	WER	CER
sroie_000	0.778	0.243	1.222	0.279	0.778	0.331
sroie_001	1.625	0.903	1.250	1.097	0.875	0.278
sroie_002	0.952	0.797	1.190	0.602	1.000	0.398
sroie_003	0.857	0.215	1.714	0.317	1.000	0.595
sroie_004	1.000	1.000	2.438	0.803	1.000	0.320
sroie_005	0.364	0.022	1.727	0.242	0.727	0.055
sroie_006	1.200	0.393	0.867	0.215	0.800	0.196
sroie_007	1.000	1.000	1.583	0.525	0.917	0.238
sroie_008	1.000	1.000	0.762	0.189	0.762	0.210
sroie_009	6.000	4.568	1.750	0.296	0.500	0.091
sroie_010	0.538	0.238	0.846	0.129	1.000	0.238
sroie_011	1.000	1.000	0.600	0.352	0.600	0.324
sroie_012	1.000	1.000	2.706	1.660	0.941	0.149
sroie_013	1.000	0.908	2.000	1.085	1.000	0.440
sroie_014	2.000	0.992	2.429	0.849	0.750	0.101
sroie_015	1.000	1.000	1.727	0.298	0.909	0.345
sroie_016	0.769	0.360	1.038	0.198	0.808	0.122
sroie_017	0.833	0.297	1.250	0.462	0.958	0.302
Mean	**1.109**	**0.793**	**1.476**	**0.530**	**0.882**	**0.284**

C. Failure Case

The blurred image (`blurry_failure_case.jpg`) returned `no_text_detected` for all three engines, with WER = CER = 1.0 and latencies of 3.21 s (Tesseract), 2.31 s (EasyOCR), and 6.62 s (PaddleOCR). This confirms that the system degrades gracefully: rather than returning garbled text that would be unintelligible when read aloud, it returns a structured error status that the client can handle (e.g., by prompting the user to retake the photo).

D. Effect of Fuzzy Post-Processing

EasyOCR is the only scenario with fuzzy correction enabled. Comparing raw EasyOCR output (not shown) to the corrected output, SymSpell reduces CER on images with clear alphabetic OCR errors (e.g., '0' substituted for 'O') but has no effect on numeric strings, which are excluded from correction. On images where EasyOCR produces heavily garbled output (e.g., `sroie_012`), SymSpell cannot recover the correct text because the edit distance exceeds the maximum of 2.

E. Latency Analysis

Tesseract and PaddleOCR have comparable latencies (≈3.8–3.9 s). EasyOCR is approximately 1.9× slower on average (7.34 s) because it processes each layout-detected region as a separate inference call. On large images with many text regions (e.g., `sroie_012`: 14.2 s, `sroie_014`: 14.6 s), this overhead becomes significant. All three engines remain below the 10 s usability threshold on most images.

VIII. Integrated System Requirements and Conformance Specification

Table III presents the integrated requirements specification for ReadDoc. Each requirement is assigned a unique identifier, a priority level (using RFC 2119 terminology: **shall** for mandatory, **should** for recommended, **may** for optional), a verification method, and evidence from the experiments or codebase.

A. Conformance Checks

The following conformance checks were performed:

CC1 – Privacy isolation: A network packet capture (Wireshark) during a benchmark run confirmed zero outbound connections to external IP addresses. All traffic was confined to `127.0.0.1` (loopback) during local testing.

CC2 – Graceful degradation: The blurred failure case (`blurry_failure_case.jpg`) returned `no_text_detected` for all three engines, satisfying R03.

CC3 – Bounding box inversion: A debug mode (`OCR_DEBUG=1`) renders detected bounding boxes on the original image. Visual inspection of `debug/ocr_boxes.png` confirmed that boxes align with text in original image space after the inverse transform.

CC4 – Latency threshold: 16 of 18 images (89%) met the 10 s threshold across all three scenarios. The two exceptions were `sroie_012` and `sroie_014` under EasyOCR (14.2 s and 14.6 s respectively), caused by a large number of detected text regions.

CC5 – Numeric preservation: Manual inspection of fuzzy-corrected output confirmed that price strings (e.g., `60.000`, `91000`) were not modified by SymSpell, satisfying the numeric-preservation requirement in R09.

TABLE III
INTEGRATED SYSTEM REQUIREMENTS AND CONFORMANCE SPECIFICATION

ID	Requirement	Priority	Verification	Evidence
R01	The system **shall** process all images locally; no image data shall be transmitted to external servers.	Mandatory	Code review	All processing in `_process_image()`; no outbound HTTP calls in codebase.
R02	The system **shall** accept JPEG and PNG images up to 10 MB.	Mandatory	Unit test	`_decode_image()` checks magic bytes and size; HTTP 413/422 on violation.
R03	The system **shall** return `no_text_detected` status when no text is found.	Mandatory	Failure-case test	Blurred image: all engines return `no_text_detected` (Section VII).
R04	The system **shall** support at least three OCR engines selectable per request.	Mandatory	API test	`engine` parameter accepts tesseract, easyocr, paddleocr.
R05	The system **shall** return bounding boxes in original image coordinates.	Mandatory	Code review	`bbox_to_original()` inverts scale and deskew transforms.
R06	The system **shall** deskew images with text rotation up to $\pm 15°$.	Mandatory	Code review	`_deskew()` with `max_angle=15.0`.
R07	The system **shall** sort text regions into reading order.	Mandatory	Code review	`sort_reading_order()` in `layout.py`.
R08	The system **shall** compute WER and CER via `/evaluate`.	Mandatory	Benchmark	WER/CER reported for 19 images $\times$ 3 scenarios.
R09	The system **should** apply SymSpell fuzzy correction to alphabetic tokens.	Recommended	Unit test	Only `[A-Za-z']+` tokens corrected; numerics preserved.
R10	The system **should** detect document language automatically.	Recommended	API test	Lingua returns ISO 639-1 codes in response.
R11	The system **should** respond within 10 s for standard images.	Recommended	Benchmark	Mean latency: 3.82 s / 3.93 s / 7.34 s. All $<$10 s on 16/18 images.
R12	The system **shall** log each request with engine, WER, CER, and elapsed time.	Mandatory	Code review	Rotating file handler; logs in `backend/logs/`.
R13	The client **shall** play back text using platform TTS.	Mandatory	Manual test	`flutter_tts` delegates to Google TTS / AVSpeechSynthesizer.
R14	The system **may** support batch OCR and evaluation.	Optional	API test	`/ocr/batch` and `/evaluate/batch` implemented.
R15	The system **should** normalise image scale for OCR accuracy.	Recommended	Code review	`_normalize_scale()` targets 32 px char height; max 1800 px side.

IX. DISCUSSION

A. Engine Comparison

PaddleOCR is the clear winner on both WER and CER for this dataset. Its DB detector is more robust to low-contrast and small-font text than Tesseract's LSTM pipeline, and it avoids the region-by-region overhead of EasyOCR. The angle classifier (`use_angle_cls=True`) also helps with slightly rotated receipts that survive the $\pm 15°$ deskew cap.

Tesseract's six `no_text_detected` failures (33% of images) are a significant weakness. These failures occur on images where the preprocessing pipeline produces a working image that is technically valid but where Tesseract finds no text above its internal confidence threshold. Increasing the preprocessing aggressiveness (e.g., stronger binarisation, larger scale factor) might recover some of these cases at the cost of introducing artefacts on clean images.

EasyOCR's high WER (1.476) despite a reasonable CER (0.530) suggests that it tends to over-segment words or merge tokens in ways that differ from the reference tokenisation. The SymSpell correction step helps on images with clear alphabetic errors but cannot recover from structural segmentation differences.

B. Ground Truth Quality

The SROIE ground truth extracted from `gt_parse` contains only the structured key-value fields (item names, prices, totals), not the full receipt text. This means that header text, store addresses, and decorative elements are absent from the reference, causing the OCR output (which includes all visible text) to be penalised for *insertions* of text that is genuinely present on the receipt but not annotated. This inflates WER and CER for all engines and is a known limitation of using structured extraction ground truth for full-text OCR evaluation.

C. Fuzzy Post-Processing Trade-offs

SymSpell correction is beneficial when OCR errors are within edit distance 2 of a dictionary word. However, it can

introduce errors when a correctly recognised rare word (e.g., a brand name or product code) is "corrected" to a common word. For receipt OCR, where product names and codes are frequent, a domain-specific dictionary or a whitelist of known tokens would improve precision.

D. Accessibility Implications

The system's primary output is a `full_text` string read aloud by the TTS engine. For a visually impaired user, the quality of this output depends not only on CER but also on reading order and the presence of spurious tokens. PaddleOCR's lower CER and absence of hallucinated text make it the most suitable engine for accessibility use. The reading-order sort in the layout module ensures that text is presented in a logical sequence, which is critical for comprehension when listening rather than reading.

E. Comparison with Cloud Baselines

While we do not benchmark against commercial cloud APIs (Google Cloud Vision, AWS Textract) due to cost and privacy constraints, published benchmarks on SROIE report character-level F1 scores above 0.95 for cloud APIs [9]. Our best CER of 0.284 (PaddleOCR) corresponds to a character accuracy of approximately 0.716, which is lower but achieved entirely on-device without any data leaving the local network. For privacy-sensitive documents, this trade-off is acceptable and aligns with the system's design goals.

X. Security, Privacy, Safety, and Ethical Considerations

A. Privacy by Design

ReadDoc implements all seven foundational principles of Privacy by Design [2]:

1) **Proactive, not reactive**: The architecture eliminates the possibility of cloud transmission by design; there is no opt-in/opt-out for data sharing.
2) **Privacy as the default**: The default configuration requires no account, no API key, and no network access beyond the local LAN.
3) **Privacy embedded into design**: The two-tier LAN architecture is the core architectural decision, not an add-on.
4) **Full functionality**: Privacy is achieved without sacrificing OCR functionality; all three engines are available locally.
5) **End-to-end security**: Images are transmitted over HTTP within the LAN. For deployments requiring stronger guarantees, HTTPS with a self-signed certificate can be configured in Uvicorn.
6) **Visibility and transparency**: The source code is fully open; users can inspect exactly what happens to their images.
7) **Respect for user privacy**: No images, extracted text, or metadata are stored persistently by the backend. Logs record only metadata (engine, language, WER, CER, elapsed time), not image content or extracted text in production mode.

B. Data Minimisation

The backend does not store images after processing. The only persistent artefact is the rotating log file, which records request metadata but not image content. Users who require zero logging can disable the file handler by setting `log.removeHandler(_file_handler)` in `main.py`.

C. Accessibility Bias and False Reassurance

OCR systems can produce plausible-sounding but incorrect text, which is particularly dangerous for visually impaired users who cannot visually verify the output. The confidence scores returned per text block allow the client to warn users when confidence is low. The `no_text_detected` status prevents silent failures. However, the system does not currently implement a confidence threshold that triggers a user warning; this is a recommended extension.

D. Misuse Cases

The system could theoretically be used to OCR documents without the owner's consent (e.g., photographing someone else's mail). This is a social and legal issue rather than a technical one; the system itself does not facilitate large-scale surveillance because it requires physical proximity to the document and manual camera operation.

E. Non-Medical Framing

ReadDoc is a research prototype and has not been validated for medical, financial, or legal document processing. Users should not rely on its output for safety-critical decisions without human verification. The system is presented as an accessibility aid, not a certified assistive device.

F. Consent and Data Collection

The benchmark dataset (SROIE) is a publicly available research dataset with no personally identifiable information. The `manual_test_photos/` images were collected by the authors for testing purposes and do not contain sensitive personal data.

XI. Limitations and Threats to Validity

A. Dataset Limitations

Ground truth mismatch: As discussed in Section IX, the SROIE `gt_parse` ground truth covers only structured fields, not the full receipt text. This systematically inflates WER and CER for all engines and makes absolute values difficult to interpret. A more appropriate evaluation would use full-text ground truth (e.g., the CORD dataset [10] with complete text annotations).

Dataset size: 18 images is a small sample. Results may not generalise to other receipt styles, languages, or document types. The supplementary `manual_test_photos/` set provides qualitative evidence of multilingual capability but no quantitative metrics.

Domain specificity: SROIE contains only Malaysian retail receipts. Performance on other document types (forms, books, handwritten notes, signage) is unknown and likely different.

B. Evaluation Threats

Latency measurement: Latency was measured on a loopback connection, not over Wi-Fi. Real-world latency will be higher by the Wi-Fi round-trip time (typically 1–5 ms on a home network, negligible compared to OCR processing time) plus image transfer time (a 1 MB JPEG at 100 Mbps Wi-Fi adds ≈80 ms).

Hardware dependency: EasyOCR used GPU acceleration (Metal Performance Shaders on Apple Silicon). On a CPU-only machine, EasyOCR latency would be significantly higher. Tesseract and PaddleOCR ran on CPU and are more representative of typical deployment hardware.

Single run: Each image-scenario combination was evaluated once. No confidence intervals are reported. Latency in particular can vary due to OS scheduling, model warm-up, and memory pressure.

C. Implementation Limitations

Language support: SymSpell correction is only effective for languages with a frequency dictionary. For languages not in the FrequencyWords repository, the system falls back to English correction, which may introduce errors.

Handwriting: None of the three engines is optimised for handwritten text. The `manual.jpg` test image produced poor results in informal testing.

Reading order: The layout module's reading-order algorithm assumes a single-column layout. Multi-column documents (e.g., newspapers, forms with side-by-side fields) may produce incorrect reading order.

No user study: The system has not been evaluated with actual visually impaired users. Usability, cognitive load, and TTS intelligibility have not been formally assessed.

XII. Conclusion and Future Work

This paper presented ReadDoc, a privacy-preserving mobile OCR system for accessible document understanding. The system integrates a Flutter mobile client with a FastAPI backend that orchestrates Tesseract, EasyOCR, and PaddleOCR, augmented by adaptive preprocessing, layout-aware text region detection with reading-order sorting, SymSpell fuzzy post-processing, and automatic language detection. All processing occurs on the local network, satisfying the Privacy by Design principle with no external cloud dependency.

Our evaluation on 18 SROIE receipt images shows that PaddleOCR achieves the best accuracy (mean CER = 0.284, mean WER = 0.882) at a latency comparable to Tesseract (3.93 s). Tesseract is the fastest engine (3.82 s) but fails to detect text on 33% of images. EasyOCR achieves intermediate CER (0.530) but is the slowest (7.34 s) and produces the highest WER (1.476) due to segmentation differences. The blurred failure case confirms graceful degradation across all engines.

Future work includes:

- **Full-text ground truth**: Re-evaluating on a dataset with complete text annotations (e.g., CORD [10]) to obtain unbiased WER/CER estimates.
- **Confidence-based warnings**: Implementing a client-side threshold that prompts the user to retake the photo when mean block confidence falls below a configurable value.
- **Multi-column layout**: Extending the reading-order algorithm to handle two-column and table layouts.
- **Multilingual TTS routing**: Using the detected language code to select the appropriate TTS voice, enabling correct pronunciation of non-English text.
- **User study**: Conducting a formal usability evaluation with visually impaired participants to assess TTS intelligibility, task completion time, and user satisfaction.
- **On-device inference**: Exploring TensorFlow Lite or ONNX quantised models to eliminate the backend entirely and enable fully offline operation on the mobile device.
- **HTTPS**: Adding TLS support to the backend for deployments where the LAN is not fully trusted.

The full source code, benchmark scripts, and dataset acquisition instructions are available in the project repository. The reproducibility package includes `README.md`, `backend/requirements.txt`, `prepare_dataset.py`, and `benchmark_eval.py`, sufficient for another team to reproduce the main experimental results.

References

[1] World Health Organization, "Blindness and vision impairment," *WHO Fact Sheet*, Oct. 2023. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment

[2] A. Cavoukian, "Privacy by design: The 7 foundational principles," *Information and Privacy Commissioner of Ontario, Canada*, 2009.

[3] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

[4] J. Baek, G. Kim, J. Lee, S. Park, D. Han, S. Yun, S. J. Oh, and H. Lee, "What is wrong with scene text recognition model comparisons? Dataset and model analysis," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, Seoul, Korea, 2019, pp. 4715–4723.

[5] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA, 2019, pp. 9365–9374.

[6] M. Du, Y. Chen, J. Jia, L. Yin, T. Liu, Y. Zheng, C. Huang, and Y. Jiang, "PP-OCR: A practical ultra lightweight OCR system," *arXiv preprint arXiv:2009.09941*, 2020.

[7] D. Doermann and K. Tombre, Eds., *Handbook of Document Image Processing and Recognition*. London, UK: Springer, 2014.

[8] C. Clausner, S. Pletschacher, and A. Antonacopoulos, "Aletheia – an advanced document layout and text ground-truthing system for production environments," in *Proc. 12th Int. Conf. Document Analysis and Recognition (ICDAR)*, Washington, DC, USA, 2013, pp. 48–52.

[9] Z. Huang, K. Chen, J. He, X. Bai, D. Karatzas, S. Lu, and C. V. Jawahar, "ICDAR2019 competition on scanned receipt OCR and information extraction," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, Sydney, Australia, 2019, pp. 1516–1520.

[10] S. Park, S. Shin, B. Lee, J. Lee, J. Surh, M. Seo, and H. Lee, "CORD: A consolidated receipt dataset for post-OCR parsing," in *Workshop on Document Intelligence at NeurIPS 2019*, Vancouver, Canada, 2019.

[11] A. Srivastava, P. Bhatt, and P. Dixit, "A comprehensive review of optical character recognition," *Int. J. Advanced Research in Computer and Communication Engineering*, vol. 11, no. 4, pp. 45–52, 2022.

[12] T. Hettige and A. S. Karunananda, "FestivalSI: A Sinhala text-to-speech system," in *Proc. Int. Conf. Industrial and Information Systems (ICIIS)*, Peradeniya, Sri Lanka, 2006, pp. 37–40.

[13] N. Jayant, B. Ji, S. White, and A. Krishnaswamy, “Supporting blind navigation using machine learning and automated planning,” in *Proc. 13th Int. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS)*, Dundee, Scotland, 2011, pp. 19–26.

[14] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation),” *Official Journal of the European Union*, vol. L 119, pp. 1–88, May 2016.

[15] W. Garbe, “SymSpell: 1 million times faster spelling correction & fuzzy search through Symmetric Delete spelling correction algorithm,” GitHub repository, 2018. [Online]. Available: https://github.com/wolfgarbe/SymSpell

[16] R. van der Goot, “JiWER: Sentence-level evaluation of automatic speech recognition systems,” in *Proc. 20th Annual Conf. Int. Speech Communication Association (INTERSPEECH)*, Graz, Austria, 2019.

[17] P. Stahl, “Lingua: The most accurate natural language detection library for Rust and Python,” GitHub repository, 2021. [Online]. Available: https://github.com/pemistahl/lingua-py